

정방향 기반 전선 객체 추출 파이프라인 설계

전달된 조건에 따라 **정방향 기반 전선(object) 추출 파이프라인**을 구축합니다. 이 파이프라인은 복수의 항공 이미지와 해당 이미지들의 **내부/외부 파라미터**(Interior/Exterior Orientation Parameters, IOP/EOP)를 이용하여 3D 포인트 클라우드에서 전선 후보 점들을 추출하고 LAS 포맷으로 분류(classification)하여 출력합니다. 아래에서는 Site A, Site B, Site C 데이터에 대한 공통 모듈 구성, 실행 순서, 및 COLMAP 연동 고려사항을 단계적으로 설명합니다.

파이프라인 개요

파이프라인의 전체 흐름은 다음과 같습니다:

- 이미지 색상 필터링:** 각 입력 이미지에서 **RGB 임계조건**($R \geq 230, G \geq 25, B \geq 178 \sim 204$)을 만족하는 픽셀만 검출합니다. 이 색 범위에 해당하는 픽셀은 전선을 나타낸다고 가정하고, 해당 픽셀들을 **전선 후보 화소**로 추출합니다. (예: 밝은 자홍색 계열로 표시된 전선 마스크)
- 화소 -> 광선 변환:** 추출된 각 픽셀에 대해 해당 이미지의 **IOP/EOP**(카메라 내부/외부 파라미터)를 사용하여 **월드 좌표계의 광선(ray)**을 계산합니다. 이는 **광선 조건의 역산**을 수행하는 과정으로, 카메라 모델을 통해 2D 픽셀을 3D 공간의 광선으로 투영합니다 ① ②. 각 픽셀은 카메라의 **투영 중심(촬영 위치)**에서 시작하여 해당 픽셀 방향으로 나아가는 하나의 광선으로 표현됩니다.
- 포인트클라우드 최근접 점 탐색:** 미리 준비된 Pix4D 출력의 **포인트클라우드(.las)**를 사용하여, 각 광선과 **가장 가까운 포인트클라우드 점**을 찾습니다. 즉, 각 광선에 대해 포인트클라우드 상에서 **점-직선 거리**가 최소인 점을 계산합니다. 이렇게 하면 해당 화소가 가리키는 3D 위치에 근접한 점을 식별할 수 있습니다. 계산 방법은 광선의 직선 방정식과 각 후보 점 간의 수직거리로 구하며, 대량의 연산은 GPU 가속(PyTorch 활용 등)으로 수행하여 효율을 높일 수 있습니다.
- 다중 영상 교차 검증:** 하나의 포인트클라우드 점이 여러 이미지의 광선들과 교차(근접)하면, 해당 점에 대한 누적 **count** 값을 증가시킵니다. 모든 이미지를 처리한 뒤 각 점마다 이 count를 합산하고, **count $\geq N$ (예: 3)** 이상인 점만 **최종 전선 후보**로 판별합니다. 여러 시점에서 일관되게 검출된 점들만 남김으로써 오검출을 줄이고 정확도를 높입니다.
- LAS 결과 저장:** 최종 전선 후보로 선택된 포인트클라우드 점들에 대하여 LAS 파일로 출력합니다. 출력 LAS의 해당 점들은 **Classification 값 14**로 설정하여 저장합니다. (LAS 표준에서 코드 14는 전선(전력선) - 도체를 의미합니다 ③.)

이러한 흐름을 통해, 칼라 필터로 검출된 전선 후보 화소가 최소 N개의 영상에서 포인트클라우드의 동일한 3D 점을 가리킬 때만 해당 점을 전선으로 최종 분류합니다.

모듈별 구성

위 파이프라인은 이해와 재사용을 쉽게 하기 위해 몇 가지 **모듈**로 나누어 구현됩니다. 각 모듈의 기능과 인터페이스는 아래와 같습니다:

1. 카메라 파라미터 파서 (Camera Parameter Parser)

- 역할:** 각 Site의 카메라 보정 결과를 읽어와 **내부 파라미터(IOP)**와 **외부 파라미터(EOP)**를 파싱합니다. IOP에는 보통 초점거리(f), 주점(principal point) 좌표, 왜곡 계수 등이 포함되며, EOP는 카메라의 **위치**(카메라 센터 좌표)와 **방향**(자세, 회전행렬 또는 쿼터니언) 정보를 담습니다 ②. 파서는 다양한 형식의 보정 데이터를 공통 구조로 변환합니다.

- **입력:** Pix4D 보정 리포트, COLMAP 출력 파일 등 (예: Pix4D .xml/calibration 파일 또는 COLMAP의 cameras.txt와 images.txt 등). Site A/B/C 모두 동일한 형식으로 제공되도록 전처리하거나, 각 형식별 분기 처리합니다.
- **출력:** 각 이미지 파일명에 대응되는 카메라 파라미터 객체 또는 딕셔너리. 예를 들어 CameraParams 클래스 인스턴스로, 속성으로 focal_length, principal_x, principal_y, distortion_coeffs, R(world->cam), C(cam_center_world) 등을 갖도록 정의합니다. 이때 모든 좌표계는 통일된 기준 (Pix4D 포인트클라우드와 동일한 월드좌표계)으로 맞춰줍니다.
- **설명:** 이 모듈을 통해 파이프라인은 Site A/B/C의 카메라 내외부 요소를 손쉽게 접근할 수 있으며, COLMAP과 Pix4D 간 좌표계 차이가 있을 경우 (예: 단위 스케일, 기준계 방향) 필요한 변환을 적용합니다. 이를 통해 이후 단계의 광선 투사에 정확한 입력이 제공됩니다.

2. 광선 생성기 (Ray Generator with Distortion Correction)

- **역할:** 주어진 이미지에서 색상 필터로 검출된 픽셀 좌표들에 대해, 각 픽셀이 카메라 중심을 지나는 3차원 광선으로 변환되도록 광선 방향 벡터를 계산합니다. 렌즈 왜곡이 있을 경우 이를 보정(언디스토션)하여 정확한 광선 방향을 얻습니다.
- **입력:**
 - 화소 좌표 목록 (예: $[(x, y), \dots]$) - 색상 조건을 만족한 픽셀들의 이미지 좌표.
 - 해당 이미지의 카메라 내부파라미터 (f, cx, cy, 왜곡계수 등) 및 외부파라미터 (R, C).
- **처리:**
 - **왜곡 보정:** 각 픽셀 좌표 (x, y)에 대하여 렌즈 왜곡 모델(예: 투습(radial) 왜곡, 접선(tangential) 왜곡)을 보정합니다. OpenCV 등의 함수나 수식으로 (x, y) -> 보정된 (x_u, y_u) (undistorted)로 변환합니다. 이 과정으로 이미지 평면에서의 이상적인 점을 얻습니다.
 - **카메라 좌표계 변환:** 보정된 픽셀 좌표를 정규화 카메라 좌표로 변환합니다. 예를 들어 $X_c = (x_u - cx) / f$, $Y_c = (y_u - cy) / f$, 그리고 가상 깊이 $Z_c = 1$ 로 두어 카메라 좌표계 방향 벡터 (X_c, Y_c, 1)를 형성합니다. 이 벡터를 단위화(normalize)하여 카메라 좌표계 기준 광선 방향으로 사용합니다.
 - **월드 좌표계로 변환:** 카메라 외부파라미터 EOP (회전행렬 R, 카메라 중심 C)를 이용하여, 카메라 좌표계의 방향 벡터를 월드 좌표계로 변환합니다. 일반적으로 월드→카메라 회전행렬이 R이라면, 광선 방향의 월드 벡터 $d_{world} = R^T * (X_c, Y_c, 1)_{cam}$ 으로 계산합니다. 광선의 시작점(origin)은 해당 카메라의 월드 좌표 C입니다.
- **출력:** 광선 객체 또는 튜플 (origin, direction)의 리스트. 각 픽셀에 대해 카메라 중심 C와 정규화 방향벡터 d_world로 정의된 광선을 반환합니다. 이 광선은 실제로 이미지 픽셀이 관측하는 공간상 모든 점들의 집합(반직선)을 나타내며, 카메라 보정 결과에 의해 정확하게 정의됩니다 ①.
- **설명:** 이 모듈을 통해 이미지 상의 후보 픽셀들이 월드 좌표계에서 어떤 방향으로 뻗어나가는지 알 수 있습니다. 이는 추후 포인트클라우드와 교차시켜 전선의 3D 위치를 찾는 기반이 됩니다. (카메라 재색선(resolution) 개념으로 보면, “각 픽셀에 대응되는 입사 광선을 결정”하는 단계라 할 수 있습니다 ①.)

3. 포인트클라우드 최근접점 탐색기 (Point Cloud Nearest Neighbor Finder)

- **역할:** 앞 단계에서 얻은 수많은 광선들 각각에 대해, **포인트클라우드 상에서 가장 가까운 점(Nearest 3D Point)**을 찾아냅니다. 이를 통해 "해당 픽셀이 가리키는 3D 대상"을 포인트클라우드에서 식별하고, 이후 다중 영상 검증에 활용합니다.

- **입력:**

- **포인트클라우드 데이터** (예: LAS 파일로부터 로드한 모든 점들의 리스트 또는 배열). 각 포인트는 (X, Y, Z) 좌표와 기타 속성(색상, 기존 분류 등)을 갖습니다.

- **광선 리스트** $[(C_i, d_i), \dots]$ - 각 이미지의 후보 화소들로부터 생성된 모든 광선들의 모음.

- **처리:** 각 광선에 대해 다음을 수행합니다:

- **최근접 후보 검색:** 광선과 포인트클라우드 점 사이의 거리를 계산하는 공식을 이용해, 광선에 가장 근접한 점을 찾습니다. 광선이 점 O (카메라 중심)와 방향 단위벡터 d 로 정의될 때, 임의의 점 P 에 대한 광선까지의 거리는 $dist = ||(P - O) - ((P - O) \cdot d) * d||$ 로 계산됩니다. 이 때 $t = (P - O) \cdot d$ 는 광선 방향으로의 투영거리입니다. 우선 모든 점 P 에 대해 t 를 계산하고 $t < 0$ (광선 뒤쪽에 위치한 점)은 제외합니다 (카메라 앞쪽의 점만 고려).

- **거리 계산:** 나머지 점들에 대해 $(P-O)$ 벡터와 방향 d 의 **벡터 외적** 또는 위 공식을 통해 거리 제곱값을 구합니다. 가장 작은 거리값을 갖는 점을 해당 광선의 최근접 점으로 선택합니다.

- **효율 향상:** 위 연산을 단순히 모든 점에 대해 반복하면 비효율적이므로, **벡터화 연산**이나 **공간 인덱싱**을 활용합니다. 예를 들어, PyTorch를 사용해 다수의 광선과 포인트 집합 간 거리를 한꺼번에 계산하면 GPU의 병렬연산으로 속도를 높일 수 있습니다. `torch.cdist` 등의 함수를 활용하면 두 집합의 모든 쌍 간 Euclidean 거리를 빠르게 구할 수 있으며, 내부적으로 행렬 곱셈 등으로 최적화됩니다. 또한 후보 점을 줄이기 위해 **KD-트리** 같은 공간 자료구조를 사용할 수 있습니다. 실제로 최근접 이웃 문제는 **KD-트리 인덱스**로 크게 가속할 수 있음이 알려져 있습니다⁴. 구현 시에는 SciPy의 `KDTree` 또는 open3d의 spatial index 등을 사용하여 각 광선의 근처 후보군만 계산하도록 최적화 가능합니다.

- **최근접 점 매칭:** 가장 가까운 점을 찾았다면, 해당 점의 ID를 기록하고 그 점에 대한 **count**를 **1 증가**시킵니다. 이 매칭 정보를 별도의 구조(예: `point_votes[point_id] += 1`)에 누적합니다. 만약 하나의 광선에 동일 최소거리로 여러 점이 걸치는 극단적 경우가 있다면, 가장 가까운 하나만 선택하거나 모두 count 증가시킬 수 있지만, 일반적으로 포인트클라우드가 촘촘하다면 유일한 최근접점이 존재하게 됩니다.

- **출력:** 각 광선이 연결된 포인트의 인덱스 리스트 (또는 딕셔너리 형태의 point->count 맵). 동시에 내부적으로 각 포인트별 누적 count를 갱신해놓습니다. 이 모듈을 완료하면, 어떤 포인트가 몇 개의 광선(=몇 개의 이미지)으로부터 지지되었는지 정보가 확보됩니다.

- **설명:** 이 단계에서는 이미지 기반 검출 결과를 3D 점과 연결짓는 **매칭**이 이뤄집니다. 수많은 연산이 필요할 수 있으므로, GPU 가속이나 C++ 확장을 통한 최적화에 신경써야 합니다. 기본 구현은 파이썬 루프와 numpy로도 가능하지만, 방대한 점군(数百万级)에서는 비효율적일 수 있으므로 앞서 언급한 기법들로 성능을 확보합니다.

4. 다중 뷰 카운트 필터 모듈 (Multi-view Count Filter)

- **역할:** 포인트별로 누적된 **관측 빈도(count)**를 이용하여 전선 후보를 최종 결정합니다. 구체적으로, 일정 **이상(N개 이상)** 이미지에서 검출된 광선들과 교차한 포인트만 **전선으로 판정**합니다.

- **입력:** 이전 모듈에서 생성된 포인트별 count 테이블 (`point_votes` 맵 또는 배열)과 임계값 **N** (예: 3). N은 최소 요구되는 교차 이미지 수로, 사용자 설정값입니다.
- **처리:** `point_votes`를 순회하거나, vectorized 연산으로 각 포인트의 count 값을 평가합니다. `count >= N` 조건을 만족하는 포인트들을 선별합니다. 이 때 N=3이라면 최소 3개 이상의 서로 다른 이미지에서 해당 점이 일관되게 전선 색으로 검출되었음을 의미합니다. 임계값이 높을수록 보수적인 필터링이 이루어지며, 낮을수록 검출률은 높아지나 오검출 위험이 커집니다.
- **출력:** **전선 후보 포인트 목록** (혹은 포인트 인덱스 집합). 이 목록에 포함된 포인트들은 신뢰도 있게 전선으로 식별된 최종 결과입니다. 이 외의 포인트들은 전선이 아닌 것으로 간주되어 배제됩니다.
- **설명:** 이 모듈은 다중 시점 검증을 통해 오검출을 제거하는 **후처리 필터**입니다. 단일 이미지에서 색이 비슷해 잘못 검출된 잡음 점들도, 다른 이미지에서는 검출되지 않으면 count=1로 걸러지게 됩니다. N 값은 데이터셋의 이미지 중첩 정도(overlap)와 품질에 따라 조정이 필요합니다. (예: 이미지가 2장만 겹치는 지역이 있다면 N=3은 과도할 수 있음)

5. LAS 작성기 (Result LAS Writer)

- **역할:** 최종 분류된 전선 포인트들을 **LAS 파일**로 저장하거나, 기존 LAS에 클래스피케이션 태그를 입혀 출력합니다. 사용자는 이 결과 LAS를 통해 전선 점들만 별도로 시각화하거나 후속 분석에 활용할 수 있습니다.
- **입력:**
 - 원본 포인트클라우드 (LAS 형식으로 메타데이터 포함).
 - 전선 후보 포인트의 인덱스 목록.
 - 출력 파일 경로, 및 지정할 **분류 코드** (Classification Value, 기본 14).
- **처리:**
 - **LAS 로드:** 라이브러리 (예: `laspy`)를 사용하여 원본 LAS 파일을 읽어옵니다. LAS 헤더와 모든 포인트 데이터를 메모리에 올립니다.
 - **분류 치환:** 각 포인트에 대해, 만약 그 인덱스가 전선 후보 목록에 있다면 해당 포인트의 Classification 필드를 14로 설정합니다. (기존 클래스 값은 덮어쓰거나, 필요하면 별도 필드에 백업 가능).
 - **LAS 저장:** 수정된 포인트 데이터를 새로운 LAS 파일로 저장합니다. 이 때 헤더의 점 개수 등 기본 메타정보를 업데이트하고, 필요한 경우 LAS 버전이나 포맷도 지정합니다.
 - **출력:** 지정 경로의 LAS 파일. 이 LAS의 분류 정보에 따라 전선으로 판정된 점들은 **코드 14 (Wire - Conductor)**로 표기되어 저장됩니다 ³. LAS 표준에 따르면 14는 전력선 도체를 의미하는 코드이므로, 표준 호환 소프트웨어에서 이 점들을 필터링하거나 색상 지정하여 쉽게 식별할 수 있습니다.
 - **설명:** 결과 LAS에는 오직 전선 후보만 포함하도록 **필요시 필터링**할 수도 있습니다. 예를 들어 전선 이외 클래스의 점들은 제거하고, 전선 점만 새로운 LAS로 내보내는 것도 한 방법입니다. 그러나 요구사항에 따라 **원본 점군 유지 + 분류 태깅** 혹은 **전선 클래스만 별도 추출** 중 선택하면 됩니다. 본 설계에서는 분류 태깅 방식을 기본으로 설명하였습니다.

모듈 간 처리 흐름 및 인터페이스

위 모듈들을 활용한 전체 처리 흐름을 순서도 형태로 정리하면 다음과 같습니다:

- 1. 카메라 파라미터 파싱:** 예를 들어 `cameras = parse_camera_params(input_calib_file)` 형태로 호출하며, `cameras`는 이미지 이름을 키로 하고 CameraParams 객체를 값으로 가지는 딕셔너리입니다. 이 단계에서 Site A, B, C 각각에 대한 카메라 보정 파일을 읽어와 메모리에 보관합니다.
- 2. 포인트클라우드 로드:** `point_cloud = load_pointcloud_las(input_las_file)` 호출로 LAS 파일을 메모리에 적재합니다. `point_cloud`는 $N \times 3$ 좌표 배열과 기타 속성으로 구성된 객체라고 가정합니다.
- 3. 이미지 반복 처리:** 각 카메라에 대해 다음을 수행합니다:
 - a. 영상 읽기 및 필터링:** `pixels = filter_pixels_by_color(image, threshold=(R≥230, G≤25, B178~204))`를 수행합니다. 이 함수는 이미지를 로드하여 RGB 조건에 맞는 픽셀 좌표 리스트를 반환합니다. (필요시 이미지 미리 로드 가능)
 - b. 광선 생성:** `rays = generate_rays(pixels, cameras[image_name])` 호출로, 앞서 얻은 픽셀 좌표들을 해당 카메라의 좌표계에서 광선으로 변환합니다. 반환된 `rays`는 (origin, direction) 튜플들의 리스트입니다.
 - c. 최근접 점 탐색 및 카운트:** 각 광선에 대해 `nearest_idx = find_nearest_point(ray, point_cloud)`를 호출하거나, 일괄처리가 가능하면 `nearest_indices = find_nearest_points_batch(rays, point_cloud)` 등을 사용합니다. 결과로 얻은 포인트 인덱스 (`nearest_idx`)에 대해 `point_votes[nearest_idx] += 1`을 수행합니다. 이 과정을 현재 이미지의 모든 rays에 대해 반복합니다.
※ 구현 최적화를 위해 (b)~(c)를 한꺼번에 벡터화할 수도 있습니다. 예를 들어 하나의 이미지에 대해 수백 개 픽셀이 검출되었다면, 이 픽셀들로부터 rays를 계산한 뒤 한 번의 대량 연산으로 해당 광선들에 대한 최근접 포인트들을 구하는 방식입니다. 이는 PyTorch에서 (R, N) vs (P, N) 크기의 행렬 간 연산이나 거리 계산으로 병렬화할 수 있습니다.
- 4. 다음 이미지 처리:** 3의 과정을 모든 이미지에 대해 수행합니다. Site A, B, C 각 사이트의 모든 이미지를 처리하면 `point_votes`에는 전체 시야에서 전선으로 추정되는 포인트들의 누적 관측 횟수가 채워져 있습니다.
- 5. 카운트 필터링:**
`wire_point_indices = [i for i, cnt in point_votes.items() if cnt >= N]`를 통해 threshold를 적용합니다. 예를 들어 $N=3$ 이라 하면, `cnt >= 3`인 포인트 `i`들만 추출합니다. 이 목록이 최종 전선 점 후보입니다.
- 6. LAS 작성:** `write_las(output_las_file, point_cloud, wire_point_indices, class_label=14)`를 호출하여 결과를 저장합니다. 구현에 따라 이 함수는 `point_cloud` 객체 내부의 분류 배열을 수정해 파일로 쓰거나, `wire_point_indices`에 포함된 점들만 추출하여 새 LAS를 생성할 수 있습니다.

위 흐름을 코드 형태로 간략히 표현하면 다음과 같습니다:

```
# 1. 카메라 파라미터 파싱
cameras = parse_camera_params(calib_path) # {image_name: CameraParams}

# 2. 포인트클라우드 로드
points, point_attrs = load_pointcloud_las("SiteA_cloud.las") # points: numpy array (N,3)

point_votes = np.zeros(len(points), dtype=int) # 포인트별 투표 count 배열

# 3. 각 이미지에 대해 처리
for img_name, cam in cameras.items():
```

```

image = load_image(img_name)
pixels = filter_pixels_by_color(image, R_min=230, G_max=25, B_range=(178,204))
rays = []
for (x,y) in pixels:
    origin = cam.C # 카메라 중심 (world coords)
    dir_cam = undistort_and_normalize(x, y, cam) # 왜곡보정+정규화 카메라 방향
    dir_world = cam.R.T @ dir_cam # 월드 방향으로 변환
    rays.append((origin, dir_world))
# 최근접 점 찾기 (벡터화 또는 루프)
nearest_idx_list = find_nearest_points_batch(rays, points)
for idx in nearest_idx_list:
    point_votes[idx] += 1

# 4. count 기반 필터링
wire_indices = [i for i, cnt in enumerate(point_votes) if cnt >= 3]

# 5. LAS 저장 (classification=14 할당)
write_las_with_classification("SiteA_wires.las", points, point_attrs, wire_indices, class_code=14)

```

위의 의사코드(pseudocode)는 각 모듈이 어떻게 연계되는지 보여줍니다. 실제 구현 시에는 모듈마다 예외 처리, 성능 개선을 위한 벡터화, 메모리 관리 등을 추가해야 합니다. 예컨대, `find_nearest_points_batch` 내부에서 PyTorch `torch.cdist`를 사용해 `rays`의 방향벡터와 `points` 간 거리를 계산함으로써 GPU 가속을 할 수 있습니다. 또한 한꺼번에 계산하는 양이 너무 큰 경우 (예: 점 1000만 개와 광선 수천 개의 조합)에는 **배치 분할** 처리가 필요합니다.

COLMAP 파이프라인 연동 및 구현 고려사항

- COLMAP 데이터 연동:** 본 파이프라인은 COLMAP 추출 카메라 파라미터와도 연동 가능하도록 설계되었습니다. COLMAP의 `images.txt`에는 각 이미지의 quaternion/translation (또는 rotation matrix)과 이미지 좌표계 → world 좌표계 변환 정보가 담겨있고, `cameras.txt`에는 내부 파라미터(focal, `k1, k2` 왜곡 등)가 있습니다. Camera Parameter Parser 모듈에서 COLMAP 파일을 읽어들이어 CameraParams 구조체에 넣으면 이후 과정은 동일합니다. 다만 Pix4D의 포인트클라우드와 COLMAP의 좌표계가 다른 경우 **좌표계 정규화**가 필요합니다. 예를 들어 Pix4D 결과가 지상좌표계(geo-referenced)라면, COLMAP 결과를 scale 및 transform하여 동일 좌표계로 변환해야 point-cloud 매칭이 정확히 이루어집니다. 이러한 변환은 보정 점 또는 Pix4D 내보낸 카메라 좌표를 이용해 계산할 수 있습니다.
- 성능 및 GPU 활용:** 기본 구현은 CPU 기반이지만, 연산량이 많은 **최근접점 탐색** 단계에서는 GPU를 활용한 벡터화 연산으로 가속하는 것을 고려합니다. 예를 들어 PyTorch의 `torch.cdist` 함수는 두 집합간의 p-노름 거리를 구할 수 있어, 광선 방향 벡터와 점 좌표간의 거리를 한 번에 계산하는 데 사용할 수 있습니다. 이를 위해 광선을 점으로 표현해야 하는데, 광선의 한 점으로 **카메라 중심과 광선을 따라 충분히 먼 한 점**(예: origin + d100m)을 취하여 두 점으로 이루어진 선분으로 근사하거나, 거리 계산 공식을 변형하여 구현할 수 있습니다. 대안으로 cuSpatial이나 faiss* 같은 라이브러리를 이용한 3D 최근접 검색도 고려 가능합니다. 데이터가 클 경우 GPU 메모리 한계를 넘지 않도록 적절히 분할하고, CPU 멀티스레딩 (예: `multiprocessing` 또는 Numpy 연산 + GIL 해제를 통한 다중 스레드 처리)도 활용합니다.
- 모듈화 장점:** 제안된 모듈 구조는 **유연성과 유지보수성**을 높입니다. 예컨대, 추후 이미지에서 전선을 검출하는 방법을 컬러 필터에서 **딥러닝 세그멘테이션**으로 변경해도, Ray Generator 이하 단계는 그대로 활용 가능합니다. 마찬가지로, 포인트클라우드 소스가 Pix4D 대신 COLMAP이나 다른 LiDAR 데이터여도, Point Cloud Finder와 이후 단계는 동일하게 적용됩니다. 각 모듈은 명확한 입력/출력 인터페이스를 가지므로, 독립적인 테스트와 개선도 수월합니다.

• **함수 인터페이스 요약:** (요구한 인터페이스 정의를 정리하면)

- `parse_camera_params(path) -> Dict[str, CameraParams]` : 보정파일을 읽어 카메라 파라미터들을 딕셔너리로 반환.
- `filter_pixels_by_color(image, condition) -> List[Tuple[int,int]]` : RGB 조건으로 픽셀 좌표 목록 반환.
- `generate_rays(pixels, cam_params) -> List[Ray]` : 픽셀 좌표와 해당 카메라 파라미터로 광선(원점+방향) 리스트 반환.
- `find_nearest_points_batch(rays, points) -> List[int]` : 광선 리스트에 대해 각각 최근접한 point index를 계산 (벡터화/배치).
- `write_las_with_classification(output_path, points, attrs, indices, class_code) -> None` : 주어진 포인트 집합에서 `indices`에 해당하는 점들의 classification을 `class_code`로 지정하여 LAS 파일 저장.

마지막으로, **전반적인 처리 흐름**도는 아래와 같이 요약될 수 있습니다:

【이미지(복수)】 → 색상 필터링 → **전선 픽셀들** → IOP/EOP 및 왜곡 보정 → **광선 집합** → 포인트클라우드 최근접 검색 → **포인트 + count** → count 필터 → **전선 포인트 선별** → LAS 저장 → **출력 LAS** (분류코드=14, 전선).

각 단계는 모듈화되어 구현되며, 상호 작용을 통해 신뢰성 있는 전선 점군 추출을 수행합니다. 이를 통해 Site A, B, C와 같은 실제 데이터셋에서 자동으로 전선 객체를 3D 포인트클라우드에서 식별하고 분류할 수 있습니다.

1 2 Camera resectioning - Wikipedia

https://en.wikipedia.org/wiki/Camera_resectioning

3 Lidar point classification—ArcMap | Documentation

<https://desktop.arcgis.com/en/arcmap/latest/manage-data/las-dataset/lidar-point-classification.htm>

4 algorithm - Nearest point to point, nearest point to line - Stack Overflow

<https://stackoverflow.com/questions/15670064/nearest-point-to-point-nearest-point-to-line>